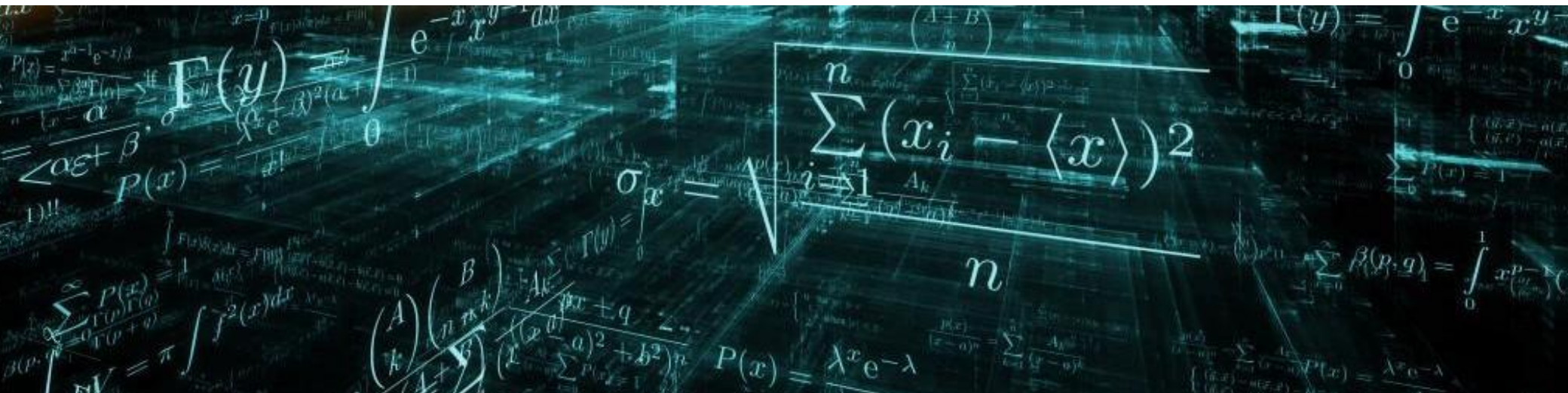


Multimedia Retrieval

Chapter 5: Semantic Search

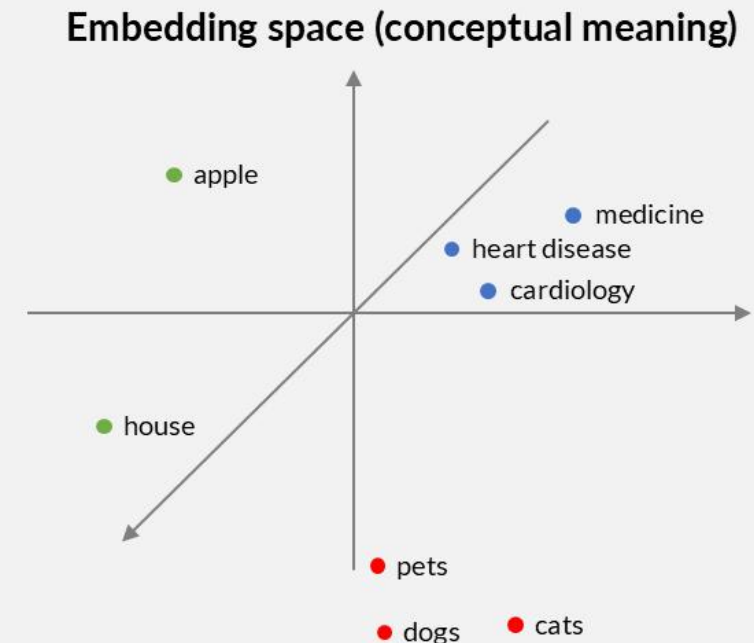
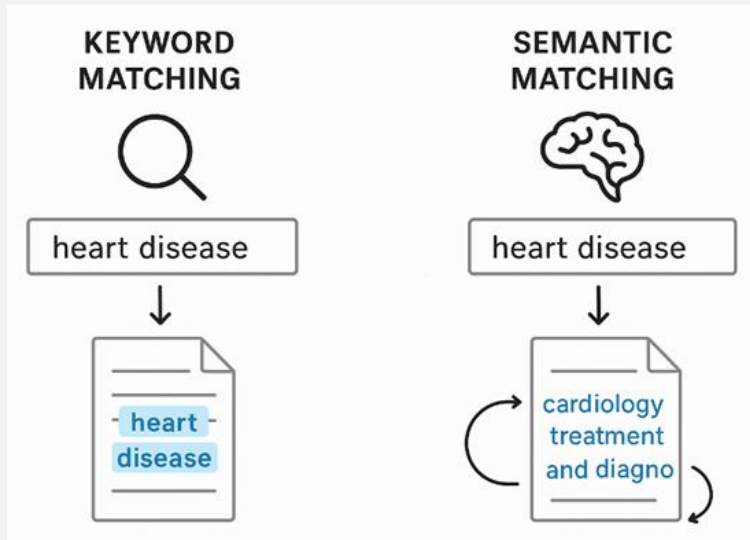
Dr. Roger Weber, roger.weber@gmail.com

- 5.1 Latent Semantic Indexing
- 5.2 Word Embeddings
- 5.3 Transformers and BERT
- 5.4 Retrieval Architectures
- 5.5 Building Search Pipelines



The vocabulary mismatch problem

- Classical retrieval treats terms as **independent symbols**
 - "heart" matches "heart", never "cardiac"
 - stemming and ontologies help at the surface, but are manual, language-specific, and brittle
- A query for "heart disease" should find myocardial infarction, even with **no shared words**
- **Semantic search:** represent text as dense vectors where geometric proximity encodes meaning
 - two terms that never co-occur can still be recognized as related
 - the shift is from discrete term matching to continuous meaning matching



Where semantic search fits: the retriever-ranker blueprint

- Semantic search mainly improves **recall**: it finds matches keyword methods miss
 - cost 1: dense vectors have no inverted index, so search is expensive (next chapter)
 - cost 2: a wider net lowers precision, so a reranker is usually needed
- BM25 is **not replaced**; production systems are hybrid
- One blueprint across all pipelines
 - a **retriever** selects candidates efficiently (recall-focused)
 - a **ranker** reorders them with deeper analysis (precision-focused)
 - what changes is only how each component is implemented



Latent Semantic Indexing

- Term independence limits recall: "automobile maintenance" will not match "car repair" without manual query expansion
- **LSI**: project the sparse, high-dimensional term-document space into a compact latent space
 - key insight: terms used in similar document contexts are related, even if they never co-occur
 - reducing dimensionality forces co-occurrence patterns to merge into **latent topics**
- Built on the **Singular Value Decomposition** of the term-document matrix A (m terms $\times$ n documents)

$$A = U S V^T$$

ID	Title
c1	Human machine interface for Lab ABC computer applications
c2	A survey of user opinion of computer system response time
c3	The EPS user interface management system
c4	System and human system engineering testing of EPS
c5	Relation of user-perceived response time to error measurement
m1	The generation of random, binary, unordered trees
m2	The intersection graph of paths in trees
m3	Graph minors IV: Widths of trees and well-quasi-ordering
m4	Graph minors: A survey

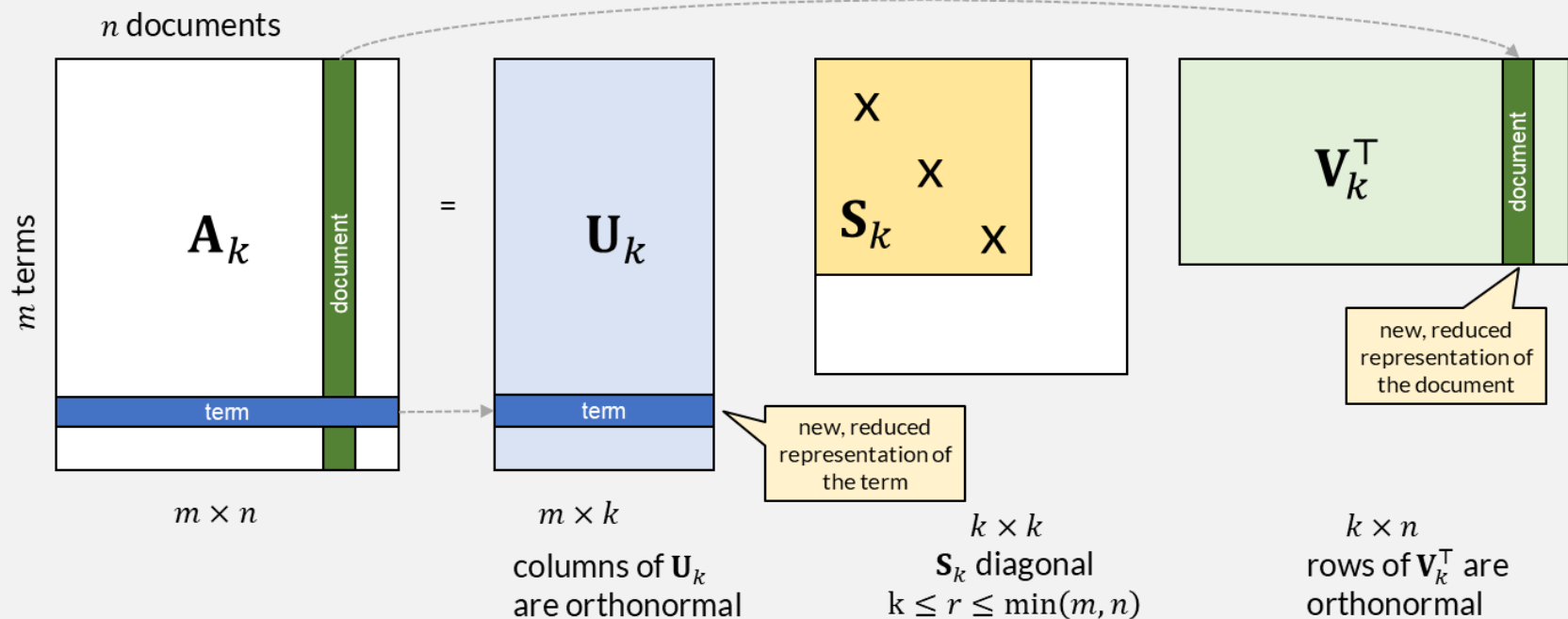
Truncated SVD and folding in

- Keep only the **k largest singular values**: the best rank- k approximation of A
 - each of the k dimensions is a latent topic not term
 - storage drops from $m \times n$ to $(m + n + 1) \times k$; typical k is 100 to 300

$$A_k = U_k S_k V_k^T$$

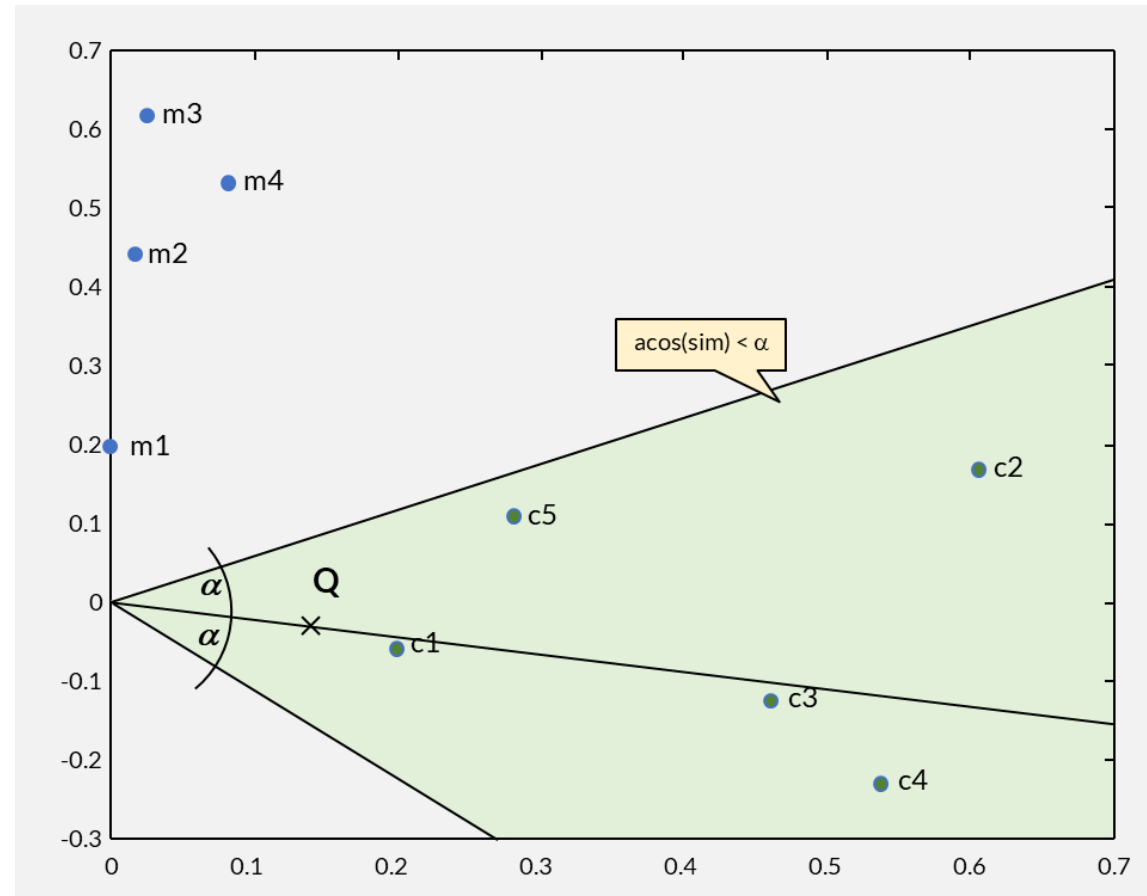
- **Folding in**: project a new document or query into the latent space without recomputing the SVD
 - queries are treated as short documents, then ranked by cosine similarity

$$\bar{d}^T = d^T U_k S_k^{-1}$$



Worked example: the Deerwester collection

- 9 documents: five on human-computer interfaces (c1-c5), four on graph algorithms (m1-m4)
- Query "human computer interaction", folded to 2D: $q = (0.138, -0.028)$
- Ranking by cosine: $c3 (0.997) > c1 (0.997) > c4 (0.979) > c2 (0.895) > c5 (0.846)$; every m-document scores below zero
 - c3 ranks top despite containing **neither** "human" nor "computer"
 - SVD linked its terms (EPS, user, interface, system) through co-occurrence
- Topic 1: system, user, EPS, time, response;
Topic 2: graph, trees, minors, survey



► [Demo: ch05-01-latent-semantic-indexing.ipynb](#)

What LSI established, and where it falls short

- The lasting principle: **dimensionality reduction discovers latent structure** that connects terms with similar usage, even when they never co-occur. Every method that follows builds on this.
- Five limitations held back adoption and motivate the rest of the chapter
 - **no inverted index**: projected vectors are dense, so every query compares against all documents
 - **corpus-specific**: topics depend on this collection; an IT-trained model fails on biomedical text
 - **static vocabulary**: new terms are only approximated through existing topics
 - **no sub-word awareness**: cannot handle morphological variants or misspellings
 - **linear**: SVD captures linear relations only; metaphor and context-dependent meaning are out of reach

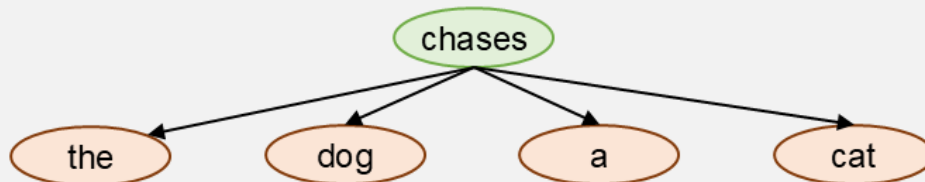
Word Embeddings

- Map each word to a dense vector (typically 100 to 300 dims) where words in similar contexts get similar vectors
- Unlike LSI, learn from **local context windows**, not a global matrix
 - streams through the corpus one window at a time: linear in corpus size
 - transfers across collections at the language level
- The **distributional hypothesis**: a word's meaning is defined by the company it keeps (Firth, 1957)
 - Word2Vec operationalizes it with a shallow neural network and self-supervision

Aspect	LSI	Word embeddings
Unit	Documents	Words
Context	Global co-occurrence	Local windows (5-10)
Training	SVD on matrix	Gradient descent
Vocabulary	Fixed	Sub-word variants (fastText)
Transfer	Corpus-specific	Language-level
Scaling	Full matrix in memory	One window at a time

Word2Vec: Skip-Gram

- **Skip-Gram** predicts the context words from a center word
 - "the dog chases a cat", center "chases", window $m=2$: predict each of the 4 neighbors independently
- Each word has two vectors: v when center, u when context
 - the inner product $u_s \cdot v_c$ scores how likely w_s appears near w_c
 - **softmax** turns scores into a probability over the whole vocabulary
- Train by minimizing the negative log-likelihood over all windows (gradient descent)

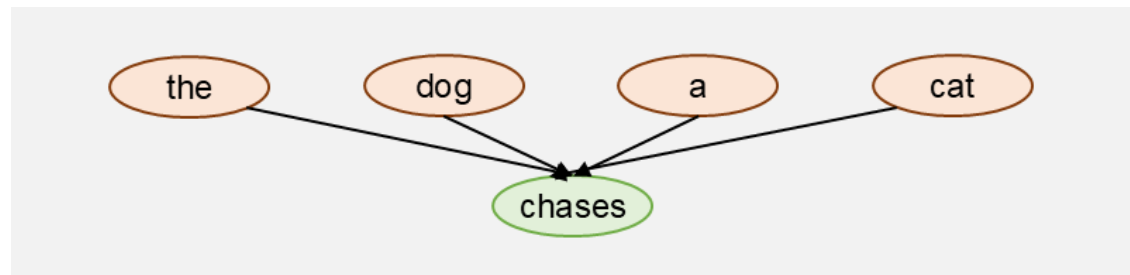


$$P(w_s | w_c) = \frac{\exp(\mathbf{u}_s^\top \mathbf{v}_c)}{\sum_{i \in \mathbb{T}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}$$

$$\mathcal{L} = - \sum_{i=m+1}^{n-m} \sum_{\substack{j=i-m \\ j \neq i}}^{i+m} \log P(w_j | w_i)$$

Word2Vec: CBOW

- **CBOW** reverses the direction: predict the center word from the averaged context vectors
 - one training sample per window, versus $2m$ for Skip-Gram
- Same softmax structure, averaged context replaces the single center vector
- Practical differences (quality is comparable; neither dominates)
 - **training speed**: CBOW is faster (one sample per window)
 - **rare words**: Skip-Gram wins (2m updates per occurrence)
 - **frequent words**: CBOW smooths noise by averaging



$$P(w_c \mid context) = \frac{\exp(\mathbf{u}_c^\top \bar{\mathbf{v}})}{\sum_{i \in \mathbb{T}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}})}$$

$$\bar{\mathbf{v}} = \frac{1}{2m} \sum_{\substack{k=-m \\ k \neq 0}}^m \mathbf{v}_{c+k}$$

Training Skip-Gram in PyTorch

- The **Embedding** layer replaces explicit one-hot vectors: a fast row lookup by token ID
 - first matrix holds center vectors v_i , the Linear layer maps back to vocabulary logits
- **CrossEntropyLoss** combines softmax and negative log-likelihood in one op
- The batch loader emits center-context pairs from overlapping windows
 - CBOW reuses the loop: pass all $2m$ context IDs, average them in forward

```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size: int, embedd_size: int):
        super().__init__()
        self.embeddings = nn.Embedding(num_embeddings=vocab_size,
                                       embedding_dim=embedd_size)
        self.linear = nn.Linear(in_features=embedd_size,
                                out_features=vocab_size)

    def forward(self, inputs):
        x = self.embeddings(inputs)
        x = self.linear(x)
        return x

model = SkipGramModel(vocab_size=len(vocab), embedd_size=100)
optimizer = optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()

for inputs, labels in batch_loader_skipgram(text_corpus):
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = loss_fn(outputs, labels)
    loss.backward()
    optimizer.step()
```

GloVe: global co-occurrence ratios

- GloVe (Pennington et al., 2014) learns from a **global** co-occurrence matrix, not streamed windows
- The insight: co-occurrence **ratios** reveal relationships better than raw counts
 - probe each word pair against a word w_k : large ratio means related to the first, small to the second, near 1 if both or neither
 - "solid" favors ice (8.9), "gas" favors steam (0.085), "water" and "fashion" sit near 1
- Fit vectors so the inner product approximates log co-occurrence, weighted to damp frequent pairs

probe w_k	P(k given ice)	P(k given steam)	ratio
solid	1.9e-4	2.2e-5	8.9
gas	6.6e-5	7.8e-4	0.085
water	3.0e-3	2.2e-3	1.36
fashion	1.7e-5	1.8e-5	0.96

$$J = \sum_{i,j=1}^{|\mathbb{T}|} f(X_{ij})(\mathbf{u}_i^\top \mathbf{v}_j + b_i + c_j - \log X_{ij})^2$$

fastText: sub-words and negative sampling

- The center word is split into **character n-grams**, and its vector is their sum
 - "chases" = <ch, cha, has, ase, ses, es>; context words keep full-word vectors
 - gives morphological generalization (chasing, chased share n-grams) and handles unknown words
- Replaces the full-vocabulary softmax with **negative sampling**
 - make the dot product large for observed pairs, small for k sampled negatives
 - negatives drawn by frequency^(3/4), over-representing rare words
- Sub-word lengths can be fixed (3-6) or variable (BPE, WordPiece)

$$v_{chases} = \sum_{g \in \mathcal{Z}} z_g = z_{<ch} + z_{cha} + z_{has} + z_{ase} + z_{ses} + z_{es}$$

"<" and ">" mark word boundaries, so prefixes, suffixes, and mid-word n-grams stay distinct

Emergent structure, and the polysemy limit

- Trained on large corpora, embeddings show emergent structure
 - **clustering**: related words occupy the same region (animals, meals, cities)
 - **analogies**: Paris - France + Germany is close to Berlin
 - **odd-one-out**: the word farthest from the group centroid
- Use **cosine**, not raw dot product: the vector norm tracks frequency and polysemy
- Hard limit: **one vector per word, regardless of context**
 - "bank" is identical whether financial or riverside; static embeddings cannot resolve it

```
import gensim.downloader
wv = gensim.downloader.load('glove-wiki-gigaword-100')

# Most similar to "cat"
wv.most_similar("cat", topn=3)
# [('dog', 0.88), ('rabbit', 0.74), ('cats', 0.73)]

# Analogy: Paris - France + Germany = ?
wv.most_similar(positive=["paris", "germany"],
                negative=["france"], topn=1)
# [('berlin', 0.88)]

# Odd one out
wv.doesnt_match(["bird", "dog", "cat", "town"])
# 'town'
```

► [Demo: ch05-02-word-embeddings.ipynb](#)

From words to sentences: pooling is not enough

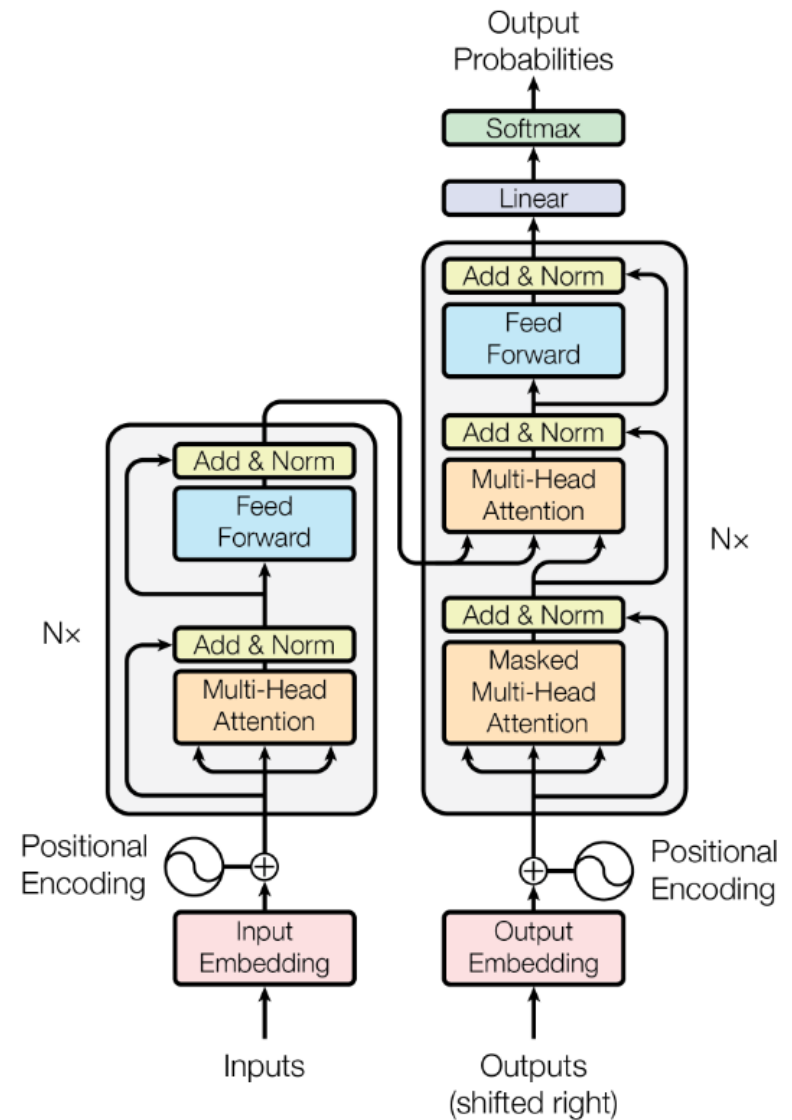
- Retrieval needs **one fixed-length vector** per query or document, not a bag of word vectors
- The naive fix is pooling word embeddings
 - **average**: the centroid of token vectors
 - **max**: the strongest activation per dimension
- Why both fail for retrieval
 - averaging dilutes: in a 500-word doc each token is 0.2%; stop words drag the centroid to generic directions
 - max discards relationships: "dog bites man" equals "man bites dog"
 - neither encodes word order or long-range dependencies

$$e_{avg} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$e_{max,j} = \max_{1 \leq i \leq n} x_{i,j}, \quad j = 1, \dots, d$$

Transformers and BERT

- Optional background: the applications in the next section only need the **outcomes**, so a reader focused on retrieval can skim this
- Word embeddings leave three problems for neural models
 - **fixed vocabulary**: no vector for unseen words (fastText's n-grams are not linguistically motivated)
 - **variable vocabulary size**: a 500k-word input layer must be rebuilt to change languages
 - **narrow context**: a 5-10 word window cannot link information across sentences
- The fixes: sub-word tokenization, learned embeddings, and self-attention



source: Attention Is All You Need

Sub-word tokenization: BPE, WordPiece, SentencePiece

- Fix the vocabulary size regardless of corpus: split text into pieces from a compact vocabulary (30k-250k)
 - any unseen word decomposes into known sub-word pieces
- **BPE**: start from characters, repeatedly merge the most frequent adjacent pair
- **WordPiece**: merge by association (PMI), mark word-internal pieces with "##"
- **SentencePiece**: operate on the raw byte stream (spaces included), language-independent; supports Unigram
- Two meanings of "embedding": the input layer (here) versus the output passage vector (retrieval)

Algorithm	Used by	Vocab	Merge criterion
WordPiece	BERT, MiniLM	30k	PMI (association)
BPE	GPT, Mistral, Qwen	32k-160k	Frequency
SentencePiece (Unigram)	XLNet, RoBERTa, BGE-M3	250k	Corpus likelihood

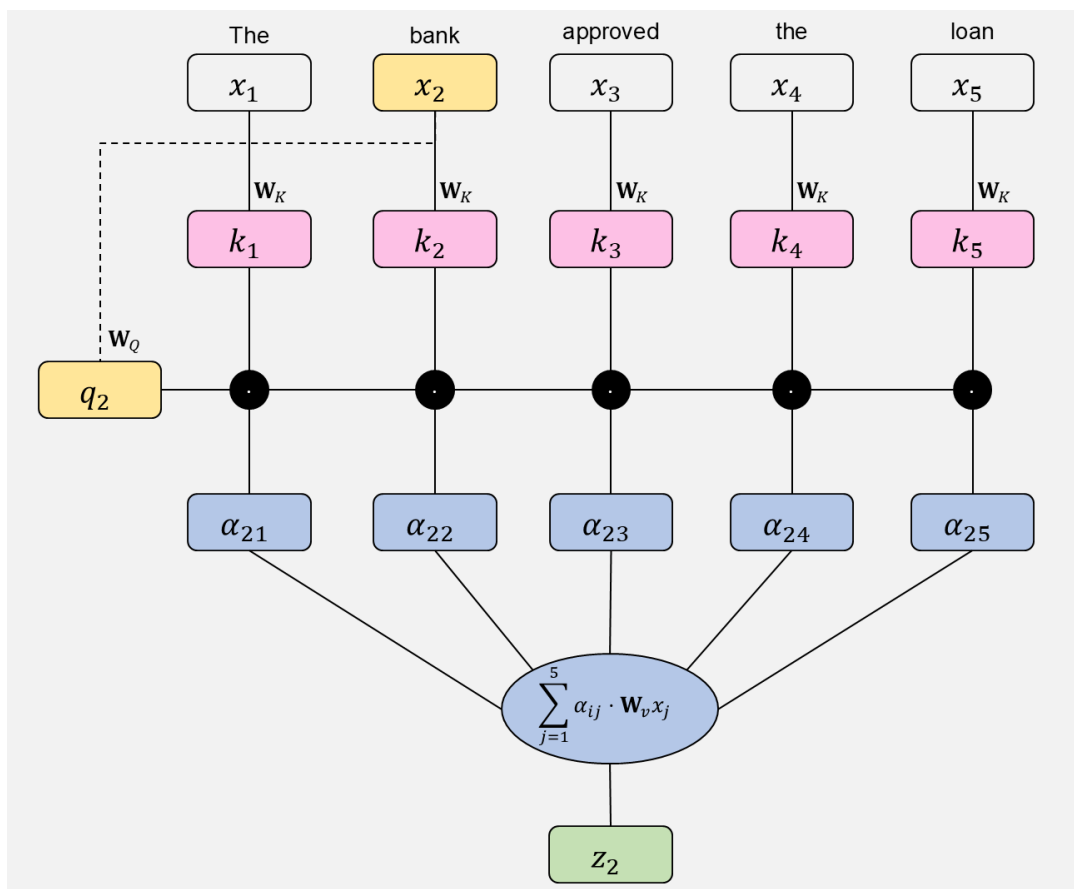
► [Demo: ch05-03-tokenization.ipynb](#)

Self-attention

- Every token attends to every other token in **parallel**, replacing sequential RNNs
 - solves both RNN problems: full parallelism, and no degradation over distance
- Three steps: **project** each token embedding x_i to a query, key, and value via learned matrices W_Q, W_K, W_V ; **score** query against all keys (scaled dot product, softmax); **aggregate** values by those weights
- The output z_i has the same dimension as the input, but is now **context-aware**
 - "bank" in "the bank approved the loan" attends to "approved" and "loan"

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

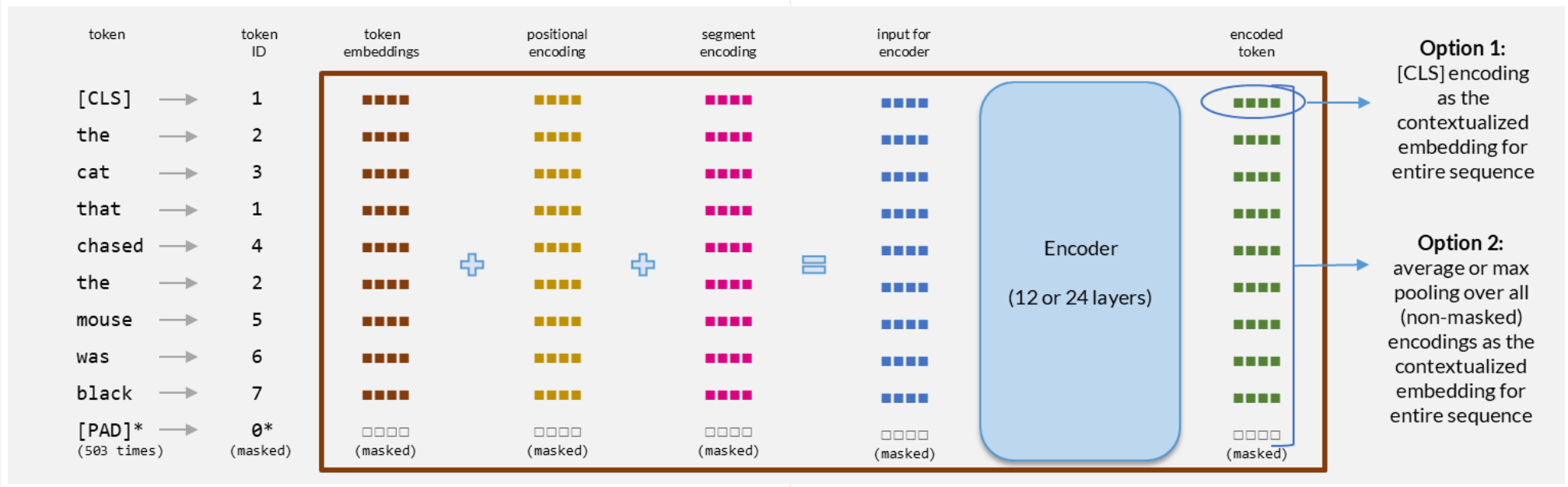
$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



The transformer and BERT

- One transformer layer: multi-head attention, then **Add & Norm**, then a **feed-forward network**, then Add & Norm
 - the FFN's ReLU/GELU adds the non-linearity; residual connections let layers stack 12 to 96 deep
- Output: 768-dim (Base) contextual vectors; the same word gets different vectors by context

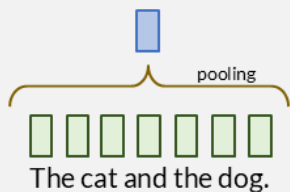
- **BERT (2019)** is the transformer **encoder**: bidirectional context, suited to understanding
 - WordPiece tokens, learned positions, segment encodings, framed by [CLS] and [SEP]
 - pre-trained with masked language modeling and next-sentence prediction



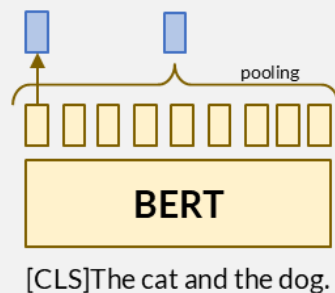
Retrieval Architectures

- Raw BERT vectors are **not** good sentence embeddings
 - [CLS] encodes the next-sentence objective, not a semantic summary
 - using it directly ranks worse than averaging Word2Vec vectors; mean-pooling is only marginally better
- Three architectures turn contextual tokens into comparable representations
 - **bi-encoder** (SBERT): encode separately, compare vectors
 - **cross-encoder**: encode jointly, score the pair
 - **late interaction** (ColBERT): per-token vectors, MaxSim scoring
- They trade off along one axis: quality versus cost and latency

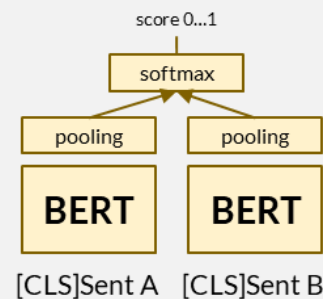
1) Pooling of token embeddings



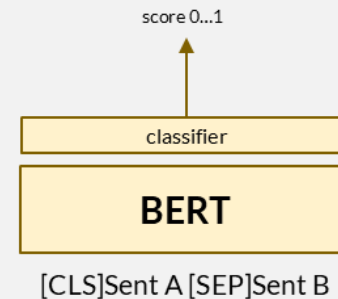
2) Pooling of encoder



3) Bi-encoder / dual-encoder

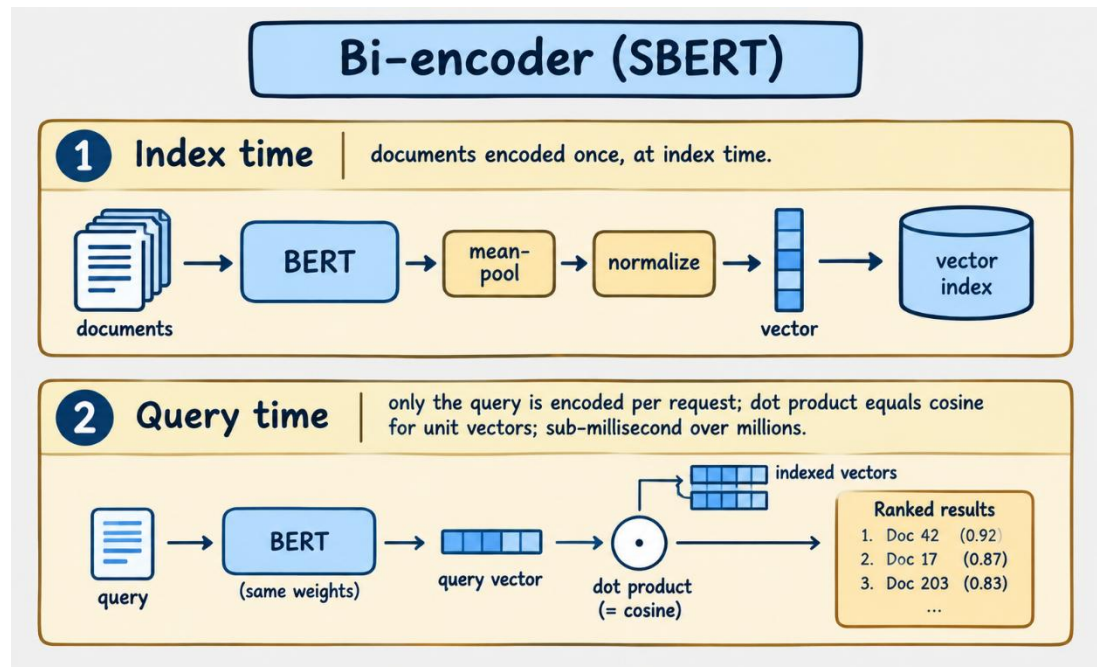


4) Cross-encoder



Bi-encoders: Sentence-BERT

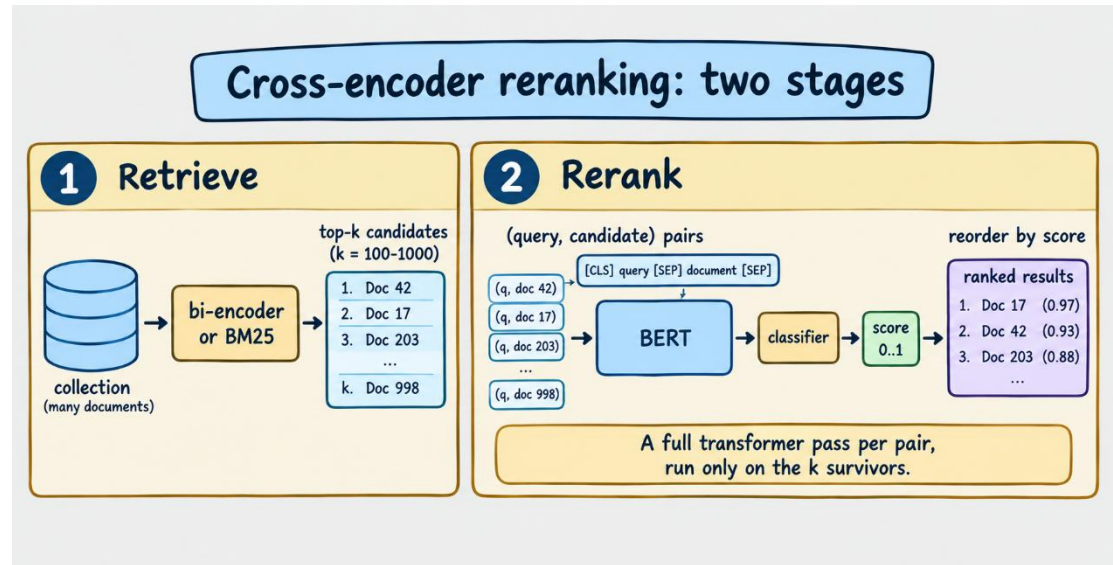
- Encode each text **independently** through the same transformer, then compare
 - pass through BERT, **mean-pool** token outputs, normalize to unit length
 - compare by dot product (equals cosine for unit vectors)
- Trained with similarity losses on labeled pairs
 - contrastive, triplet, or multiple-negatives ranking (MNRL); hard-negative mining is critical
- Why it scales: **documents are encoded once at index time**
 - at query time, encode only the query, then dot products give sub-millisecond ranking over millions



Similarity choice: match the measure the model was trained with. Cosine if the output is normalized; dot product if the norm carries signal (read the model card).

Cross-encoders and reranking

- Feed query and document **together** as one sequence: [CLS] query [SEP] document [SEP]
 - all tokens attend across both texts, so it captures fine-grained interactions bi-encoders miss
 - the [CLS] state passes through a classifier to a relevance score
- Too expensive for first-stage search: every pair needs a full transformer pass
- The standard **two-stage pipeline**
 - retrieve top-k ($k = 100$ to 1000) with a bi-encoder or BM25
 - rerank those k with the cross-encoder
 - near cross-encoder quality at near bi-encoder speed

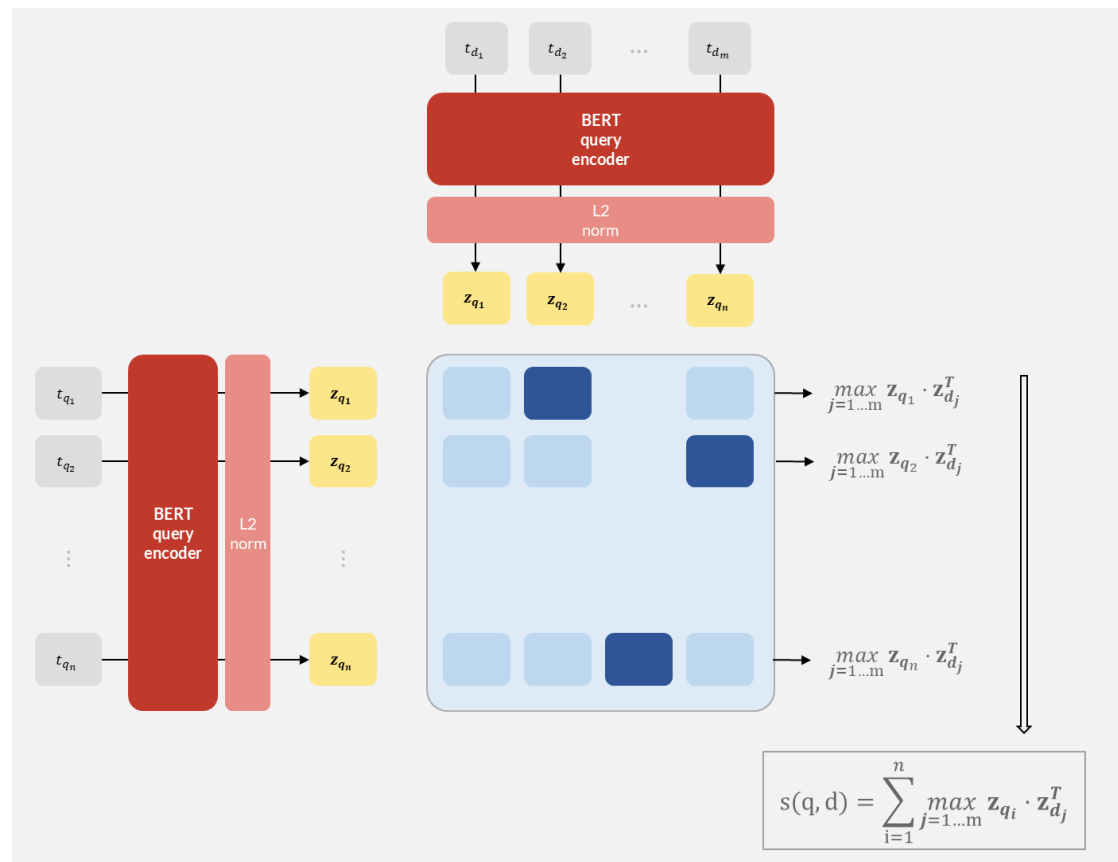


Property	Bi-encoder	Cross-encoder
Encoding	Separately	Jointly
Pre-compute	Docs at index time	Every pair at query time
Per-doc cost	$O(1)$ dot product	$O(n \cdot L)$ full pass
Semantic depth	Good	Excellent
Typical use	First-stage retrieval	Rerank top-k

Late interaction: ColBERT

- Middle ground: encode query and document **independently** (so documents pre-compute), but keep **all token vectors**
- **MaxSim**: each query token takes its best match among document tokens; sum over query tokens
 - matches are between context-aware BERT vectors, not static word vectors
 - captures context-sensitive, semantic, and partial matching
- No single-vector compression loss, so 128-dim per-token vectors suffice
 - cost is storage and search: n vectors per document, $|q| \times n$ dot products
 - reaches 95-98% of cross-encoder quality, 100-1000x faster

$$score(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \mathbf{q}_i^T \mathbf{d}_j$$



► **Demo:** [ch05-04-sentence-embeddings.ipynb](#)

Building Search Pipelines

- Semantic recall is bought at three costs classical retrieval never faced
 - **search is expensive**: no inverted index; one million docs is one million dot products
 - **precision drops**: a wider net needs cross-encoder reranking
 - **compression loses information**: one vector per long document discards detail; chunking helps but multiplies vectors
- Every design is a tradeoff among **compute cost, latency, and quality**
 - no single optimal pipeline; collection size and use case decide
- The same retriever-ranker blueprint, implemented **five ways** as requirements grow
 - A is the BM25 baseline; each later one adds a semantic component
 - B and C add reranking or dense retrieval; D fuses both; E optimizes cost at scale

#	Retriever	Ranker	Scale	Approach
A	BM25	BM25	Any	Fast keyword retrieval; misses semantic matches
B	BM25	Cross-encoder	Small-medium	BM25 candidates, cross-encoder reranks top-k
C	Bi-encoder	Cross-encoder	Small	Vector search by similarity, cross-encoder reranks
D	BM25 + Bi-encoder	Cross-encoder	Medium-large	Both retrieve, results fused, then reranked
E	Bi-encoder (compact)	Bi-encoder (full) + Cross-encoder	Very large	Cheap first stage (truncated or quantized), then refine

Choosing an embedding model (MTEB)

- The **MTEB** leaderboard compares models across retrieval, similarity, classification, clustering
 - for search, nDCG@10 on retrieval tasks is the metric that matters
- Selection criteria, each a tradeoff
 - **dimension** (256-4096): more nuance, more storage and search cost
 - **model size** (33M-8B): better embeddings, more compute to encode
 - **context** (512-32k): fewer chunks, but more compression per vector
 - **multilingual** and **instruction** support
- Scores evolve fast: always check the current leaderboard

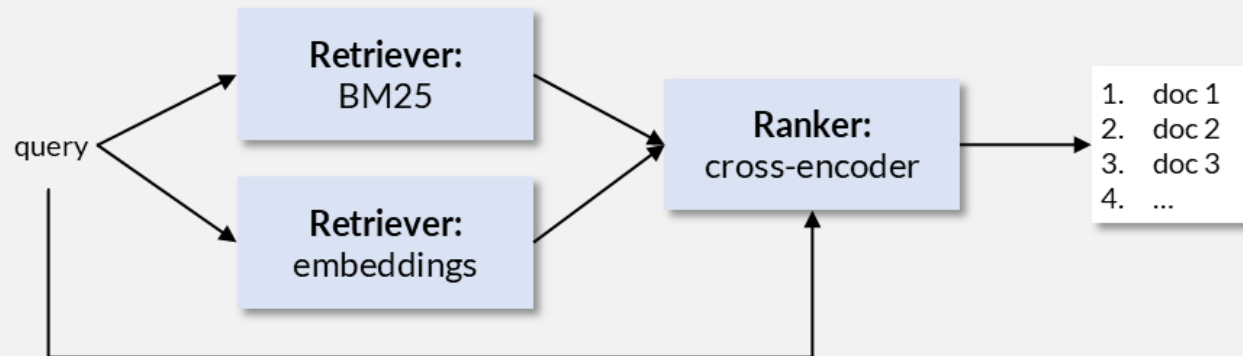
Model	Params	Dim	Context	nDCG@10
all-MiniLM-L6-v2	33M	384	512	~49
nomic-embed-text-v1.5	137M	768	8192	~55
jina-embeddings-v3	570M	1024	8192	~58
GTE-Qwen2-7B	7B	3584	32k	~61
Qwen3-Embedding-8B	8B	4096	32k	~63

Pipelines B, C, D: reranking, dense, hybrid

- **B (BM25 + reranker)**: cheapest upgrade; works when BM25 recall is enough, fails on severe vocabulary mismatch
- **C (dense retrieval)**: a bi-encoder retrieves by meaning, finds what BM25 misses, but fails on exact matches (error codes, product IDs)
- **D (hybrid)**: run both in parallel, fuse with **Reciprocal Rank Fusion**, then rerank

- **Chunking** differs by component: BM25 tolerates large chunks; bi-encoders need 256-1024 tokens; cross-encoders read longer inputs jointly

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

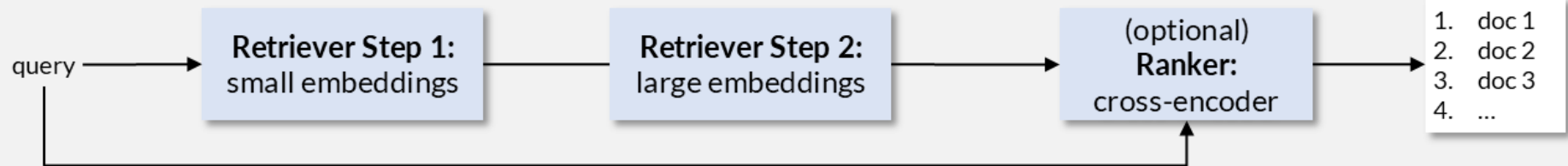


Efficiency at scale: Matryoshka and quantization (Pipeline E)

- Multi-stage retrieval: a **cheap first stage** scans the full collection, a precise full-precision stage reranks survivors
- **Matryoshka (MRL)**: train so truncated prefixes stay useful
 - first 64 dims give coarse semantics, 128 add detail, full vector is most precise
 - one model serves both stages

- **Quantization** builds the cheap stage by compressing dimensions
 - **binary** (1 bit/dim): 32x smaller, Hamming distance, 5-10% quality loss
 - **scalar** (int8): 4x smaller, under 2% loss

$$\mathcal{L}_{MRL} = \sum_{d \in \mathcal{D}} \mathcal{L}_d(\mathbf{x}_{[:d]}, \mathbf{y}_{[:d]})$$



Practical recommendations

- By collection size

Collection	Pipeline	Why
< 10k	C (bi + cross)	Brute force is fast enough
10k-1M	D (hybrid + cross)	Exact plus semantic, then refine
> 1M	E (multi-stage)	Keep latency manageable; needs ANN index

- By use case

Use case	Emphasis
Factual Q&A	Small bi-encoder, short chunks, fast reranker
Patent, legal	Large bi-encoder, hybrid, aggressive rerank
E-commerce	Hybrid mandatory (IDs need BM25)
Academic, multilingual	Multilingual model, hybrid, moderate rerank

► **Demo:** [ch05-05-building-semantic-search.ipynb](#)

Key takeaways

- Vocabulary mismatch is the motivation: related terms with no shared surface forms cannot be matched by keywords alone
- Dimensionality reduction discovers latent structure, by SVD (LSI) or neural training (embeddings)
- Static embeddings learn meaning from context but give one vector per word; pooling them into a passage vector dilutes and loses order
- Context transforms meaning: transformer encoders give the same word different vectors by its neighbors
- Bi-encoders scale retrieval (encode once, dot product); cross-encoders give precision (joint encoding, rerank top-k)
- ColBERT keeps per-token detail with MaxSim, avoiding single-vector compression
- Hybrid (BM25 + dense, fused by RRF) is robust; choose the similarity measure deliberately
- Every pipeline (A to E) is a tradeoff among compute, latency, and quality; collection size and use case decide

Further reading

- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). **Indexing by Latent Semantic Analysis**. *Journal of the American Society for Information Science*, 41(6), 391-407. [Semantic Scholar](#) The foundational paper introducing LSI and demonstrating that SVD on term-document matrices captures latent semantic structure.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). **Distributed Representations of Words and Phrases and their Compositionality**. *Advances in Neural Information Processing Systems (NeurIPS)*. [arXiv:1310.4546](#) Introduces negative sampling and phrase-level Skip-Gram, making Word2Vec practical at scale.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). **Attention Is All You Need**. *Advances in Neural Information Processing Systems (NeurIPS)*. [arXiv:1706.03762](#) Introduces the transformer architecture with self-attention, replacing recurrent models and enabling the entire subsequent generation of language models and embedding models.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**. *Proceedings of NAACL-HLT 2019*. [arXiv:1810.04805](#) Introduces bidirectional pre-training with masked language modeling, establishing the encoder architecture used by most embedding models.
- Reimers, N. & Gurevych, I. (2019). **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks**. *Proceedings of EMNLP-IJCNLP 2019*. [arXiv:1908.10084](#) The paper that made transformer-based sentence embeddings practical for retrieval by introducing the bi-encoder training framework with contrastive loss.
- Khattab, O. & Zaharia, M. (2020). **ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT**. *Proceedings of SIGIR 2020*. [arXiv:2004.12832](#) Introduces the late interaction paradigm (MaxSim), achieving near cross-encoder quality with much lower latency by retaining per-token embeddings.
- Kusupati, A., Bhatt, G., Rege, A., et al. (2022). **Matryoshka Representation Learning**. *Advances in Neural Information Processing Systems (NeurIPS 2022)*. [arXiv:2205.13147](#) Trains embedding models to produce useful representations at any truncation level, enabling multi-resolution retrieval from a single model.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). **MTEB: Massive Text Embedding Benchmark**. *Proceedings of EACL 2023*. [arXiv:2210.07316](#) Establishes the standard benchmark for evaluating embedding models across retrieval, similarity, classification, and clustering tasks.